Pennine Pizza Works: Design

Mini Project 2 · COM4401 Software Engineering Fundamentals

Marcus Weeks

Stage: 2. Design (Waterfall life cycle)

Owner: Marcus Weeks (Design and test plan design)

Builds on: Charlie's Analysis (requirements table and IPO table)

Layout follows: the tutor's worked example, *Supercar Experience Days: Solution*, Section 2

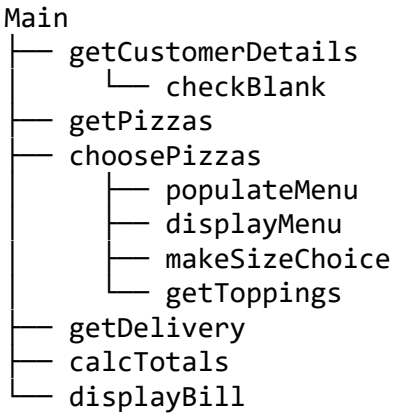
Requirements used in this design

These are Charlie’s requirements from the Analysis stage, copied as written, so every part of the design traces back to the same numbers.

ID	Requirement	Type
R1	Enter customer name.	Input
R2	Enter customer address.	Input
R3	Enter customer phone number.	Input
R4	Enter number of pizzas ordered.	Input
R5	Enter size of each pizza.	Input
R6	Enter number of extra toppings for each pizza.	Input
R7	Enter whether delivery is required.	Input
R8	Number of pizzas must be between 1 and 6.	Rule
R9	Small pizza costs £3.25.	Rule
R10	Medium pizza costs £5.50.	Rule
R11	Large pizza costs £7.15.	Rule
R12	1 extra topping costs £0.75.	Rule
R13	2 extra toppings cost £1.35.	Rule
R14	3 extra toppings cost £2.00.	Rule
R15	4 or more extra toppings cost £2.50.	Rule
R16	Apply a 10% discount when subtotal is over £20 before delivery.	Rule
R17	Add £2.50 delivery charge when delivery is requested.	Rule
R18	Calculate the cost of each pizza.	Process
R19	Calculate the order subtotal before discount and delivery.	Process
R20	Calculate discount where applicable.	Process
R21	Calculate delivery charge where applicable.	Process
R22	Calculate final order total.	Process
R23	Display customer details.	Output
R24	Display cost of each pizza.	Output
R25	Display standard pizza cost and extra topping cost.	Output
R26	Display subtotal before discount and delivery.	Output
R27	Display discount applied.	Output
R28	Display delivery charge.	Output
R29	Display final total.	Output

2.1 Decomposition: structure chart

The design splits the program into small subroutines, each doing one job. Main calls them in order.



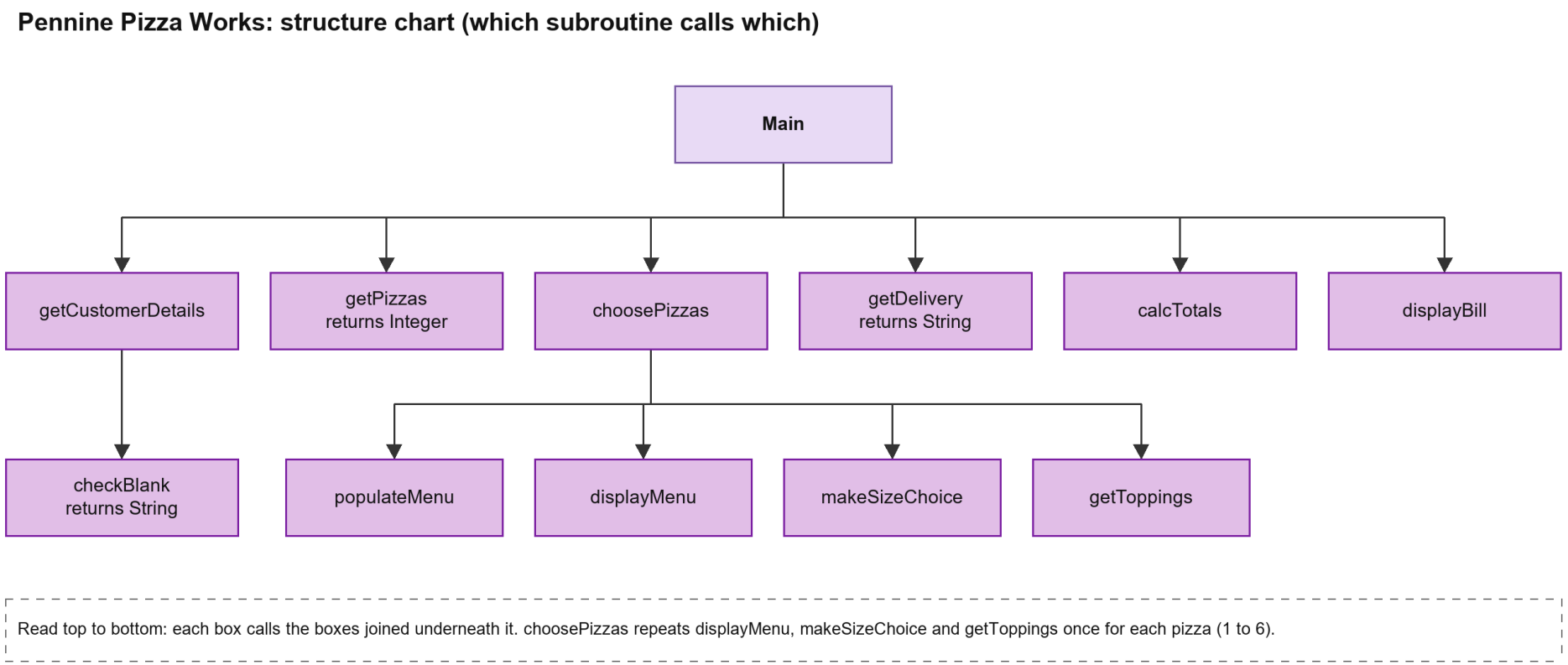


Figure 1: Structure chart (drawn from page 2 of PenninePizzaDesign.drawio)

2.2 Main flowchart design

Main sets up the storage, then calls one subroutine for each stage of the order. The comment next to each call says what it does, so Main does not need the detail. This is abstraction.

Step	Shape	What it does
1	Declare	Set up all arrays and variables (see 2.3).
2	Call	`getCustomerDetails(customerDetails)`: name, address, phone.
3	Assign	`noOfPizzas = getPizzas()`: how many pizzas, 1 to 6.
4	Call	`choosePizzas(sizeNames, sizePrice, toppingPrice, noOfPizzas, chosenSize, chosenToppings)`: size and toppings for each pizza.
5	Assign	`delivery = getDelivery()`: Y or N.
6	Call	`calcTotals(sizePrice, toppingPrice, chosenSize, chosenToppings, noOfPizzas, delivery, totals)`: subtotal, discount, delivery charge, total.
7	Call	`displayBill(customerDetails, sizeNames, sizePrice, toppingPrice, chosenSize, chosenToppings, noOfPizzas, totals)`: the itemised bill.



Figure 2: Main flowchart design (drawn from page 1 of PenninePizzaDesign.drawio)

2.3 Data plan

Data structures (arrays)

Main declares every array the subroutines share. Flowgorithm passes arrays by reference, so a subroutine can fill an array that Main owns.

Array	Type	Size	Holds	Example
customerDetails	String	3	[0] name, [1] address, [2] phone number	customerDetails[0] = “Sam Patel”
sizeNames	String	3	The size names. The index is the size ID (0 to 2).	sizeNames[2] = “Large”
sizePrice	Real	3	The price of each size, in the same order as sizeNames.	sizePrice[2] = 7.15
toppingPrice	Real	5	The topping price for 0, 1, 2, 3 and 4-or-more toppings.	toppingPrice[4] = 2.50
chosenSize	Integer	6	The size ID of each pizza ordered. Only positions 0 to noOfPizzas − 1 are used.	chosenSize[0] = 2
chosenToppings	Integer	6	The number of toppings on each pizza, in the same order as chosenSize.	chosenToppings[0] = 2
totals	Real	4	[0] subtotal, [1] discount, [2] delivery charge, [3] final total	totals[3] = 19.00
label	String	3	The word used in each prompt: “Name”, “Address”, “PhoneNo”. Declared in getCustomerDetails, not Main.	label[1] = “Address”

sizeNames and sizePrice are parallel arrays: the same index gives the name and price of the same size. chosenSize stores indexes into both, so `sizePrice[chosenSize[0]]` is the price of the first pizza.

toppingPrice uses the number of toppings as the index. Any number of 4 or more uses index 4, because the brief charges the same £2.50 for 4 or more.

Variables

Variable	Type	Used in	Purpose	Valid values
noOfPizzas	Integer	Main, returned by getPizzas	Number of pizzas in the order	1 to 6
delivery	String	Main, returned by getDelivery	Whether delivery is wanted	“Y” or “N”
counter	Integer	choosePizzas	Which pizza is being chosen, and its position in chosenSize and chosenToppings	0 to noOfPizzas − 1
choice	Integer	makeSizeChoice	The size ID typed in	0 to 2
toppings	Integer	getToppings	The number of toppings typed in	0 or more
loop	Integer	several	For loop counter, local to each subroutine	set by each loop
details	String	checkBlank (parameter and return value)	The customer detail being checked	Must not be blank

2.4 Subroutine specifications

Subroutine	Parameters	Returns	Purpose (requirements met)
getCustomerDetails	customerDetails (String array, filled by reference)	nothing	Asks for name, address and phone. Each is checked by checkBlank (R1 to R3).
checkBlank	details (String)	String	Asks again while the entry is blank, then returns it (R1 to R3).
getPizzas	none	Integer	Asks again until the number of pizzas is 1 to 6 (R4, R8).
choosePizzas	sizeNames, sizePrice, toppingPrice, chosenSize, chosenToppings (arrays), noOfPizzas (Integer)	nothing	Loads the price lists, then for each pizza shows the menu and gets the size and toppings (R5, R6).
populateMenu	sizeNames, sizePrice, toppingPrice (arrays, filled by reference)	nothing	Stores the client's price list (R9 to R15).
displayMenu	sizeNames, sizePrice (arrays)	nothing	Lists the 3 sizes with their ID and price (R5).
makeSizeChoice	chosenSize (array, filled by reference), counter (Integer)	nothing	Gets a size ID from 0 to 2 and stores it at position counter (R5).
getToppings	chosenToppings (array, filled by reference), counter (Integer)	nothing	Gets a number of toppings of 0 or more and stores it at position counter (R6).
getDelivery	none	String	Asks again until the answer is Y or N (R7).
calcTotals	sizePrice, toppingPrice, chosenSize, chosenToppings, totals (arrays), noOfPizzas (Integer), delivery (String)	nothing	Fills totals: subtotal, 10% discount if over £20, £2.50 delivery if wanted, final total (R16 to R22).
displayBill	customerDetails, sizeNames, sizePrice, toppingPrice, chosenSize, chosenToppings, totals (arrays), noOfPizzas (Integer)	nothing	Shows the customer details, each pizza with its pizza and topping cost, then the totals, all money to 2 decimal places (R23 to R29).

2.5 Algorithm: pseudocode

Main

```
SUBROUTINE Main()
    // Setup data structures and variables
    DECLARE String customerDetails[3]
    DECLARE String sizeNames[3]
    DECLARE Real sizePrice[3]
    DECLARE Real toppingPrice[5]
    DECLARE Integer chosenSize[6]
    DECLARE Integer chosenToppings[6]
    DECLARE Real totals[4]
    DECLARE Integer noOfPizzas
    DECLARE String delivery

    // ABSTRACTION: get customer details
    CALL getCustomerDetails(customerDetails)
    // ABSTRACTION: get the number of pizzas
    noOfPizzas = getPizzas()
    // ABSTRACTION: show the menu, choose size and toppings for each pizza
    CALL choosePizzas(sizeNames, sizePrice, toppingPrice, noOfPizzas,
chosenSize, chosenToppings)
    // ABSTRACTION: delivery or collection
    delivery = getDelivery()
    // ABSTRACTION: work out the totals
    CALL calcTotals(sizePrice, toppingPrice, chosenSize, chosenToppings,
noOfPizzas, delivery, totals)
    // ABSTRACTION: display the itemised bill
    CALL displayBill(customerDetails, sizeNames, sizePrice, toppingPrice,
chosenSize, chosenToppings, noOfPizzas, totals)
END SUBROUTINE
```

getCustomerDetails and checkBlank

```
SUBROUTINE getCustomerDetails(customerDetails[])
    DECLARE String label[3]
    label[0] = "Name"
    label[1] = "Address"
    label[2] = "PhoneNo"
    DECLARE Integer loop
    FOR loop = 0 TO 2
        OUTPUT "Enter " & label[loop] & " :"
        INPUT customerDetails[loop]
        // Make sure the entry is not blank
        customerDetails[loop] = checkBlank(customerDetails[loop])
    NEXT loop
END SUBROUTINE

FUNCTION checkBlank(details) RETURNS String details
    // Ask again while the entry is blank
    WHILE ToCode(Char(details & " ",0)) = 32
        OUTPUT "This cannot be blank, try again!"
        INPUT details
    END WHILE
    RETURN details
END FUNCTION
```

getPizzas

```
FUNCTION getPizzas() RETURNS Integer noOfPizzas
    DECLARE Integer noOfPizzas
    DO
        OUTPUT "Enter number of pizzas (1-6):"
        INPUT noOfPizzas
        IF noOfPizzas < 1 OR noOfPizzas > 6 THEN
            OUTPUT "Error, must be between 1 and 6!"
        END IF
    WHILE noOfPizzas < 1 OR noOfPizzas > 6
    RETURN noOfPizzas
END FUNCTION
```

choosePizzas

```
SUBROUTINE choosePizzas(sizeNames[], sizePrice[], toppingPrice[], noOfPizzas,
chosenSize[], chosenToppings[])
    DECLARE Integer counter
    counter = 0
    CALL populateMenu(sizeNames, sizePrice, toppingPrice)
    DO
        CALL displayMenu(sizeNames, sizePrice)
        CALL makeSizeChoice(chosenSize, counter)
        CALL getToppings(chosenToppings, counter)
        counter = counter + 1
    WHILE counter < noOfPizzas
END SUBROUTINE
```

populateMenu

```
SUBROUTINE populateMenu(sizeNames[], sizePrice[], toppingPrice[])
    sizeNames[0] = "Small"     sizePrice[0] = 3.25
    sizeNames[1] = "Medium"    sizePrice[1] = 5.50
    sizeNames[2] = "Large"     sizePrice[2] = 7.15
    toppingPrice[0] = 0
    toppingPrice[1] = 0.75
    toppingPrice[2] = 1.35
    toppingPrice[3] = 2.00
    toppingPrice[4] = 2.50
END SUBROUTINE
```

displayMenu

```
SUBROUTINE displayMenu(sizeNames[], sizePrice[])
    DECLARE Integer loop
    OUTPUT "Pizza Menu:"
    OUTPUT "[id] [Price] [Size]"
    FOR loop = 0 TO 2
        OUTPUT loop & " £" & ToFixed(sizePrice[loop], 2) & " " &
sizeNames[loop]
    NEXT loop
END SUBROUTINE
```

makeSizeChoice

```
SUBROUTINE makeSizeChoice(chosenSize[], counter)
    DECLARE Integer choice
    // VALIDATION: size must come from the menu
    OUTPUT "Pizza " & (counter + 1) & ": select size ID:"
    INPUT choice
    WHILE choice < 0 OR choice > 2
        OUTPUT "Invalid choice. Please enter an ID from 0 to 2"
        INPUT choice
    END WHILE
    chosenSize[counter] = choice
END SUBROUTINE
```

getToppings

```
SUBROUTINE getToppings(chosenToppings[], counter)
    DECLARE Integer toppings
    // VALIDATION: toppings cannot be negative
    OUTPUT "Pizza " & (counter + 1) & ": number of extra toppings:"
    INPUT toppings
    WHILE toppings < 0
        OUTPUT "Invalid. Toppings cannot be less than 0"
        INPUT toppings
    END WHILE
    chosenToppings[counter] = toppings
END SUBROUTINE
```

getDelivery

```
FUNCTION getDelivery() RETURNS String delivery
    DECLARE String delivery
    OUTPUT "Delivery required? (Y/N):"
    INPUT delivery
    WHILE delivery != "Y" AND delivery != "N"
        OUTPUT "Please enter Y or N"
        INPUT delivery
    END WHILE
    RETURN delivery
END FUNCTION
```

calcTotals

```
SUBROUTINE calcTotals(sizePrice[], toppingPrice[], chosenSize[],
chosenToppings[], noOfPizzas, delivery, totals[])
    DECLARE Integer loop
    DECLARE Integer t
    totals[0] = 0
    // Subtotal: size price + topping price for each pizza
    FOR loop = 0 TO noOfPizzas - 1
        t = chosenToppings[loop]
        IF t > 4 THEN
            t = 4
        END IF
        totals[0] = totals[0] + sizePrice[chosenSize[loop]] + toppingPrice[t]
    NEXT loop
    // Discount: 10% if the subtotal is over £20, before delivery
    IF totals[0] > 20 THEN
        totals[1] = totals[0] * 0.10
    ELSE
        totals[1] = 0
    END IF
    // Delivery charge
    IF delivery == "Y" THEN
        totals[2] = 2.50
    ELSE
        totals[2] = 0
    END IF
    // Final total
    totals[3] = totals[0] - totals[1] + totals[2]
END SUBROUTINE
```

displayBill

```
SUBROUTINE displayBill(customerDetails[], sizeNames[], sizePrice[],
toppingPrice[], chosenSize[], chosenToppings[], noOfPizzas, totals[])
    DECLARE Integer loop
    DECLARE Integer t
    OUTPUT "Itemised Bill"
    FOR loop = 0 TO 2
        OUTPUT customerDetails[loop]
    NEXT loop
    OUTPUT "Number of pizzas: " & noOfPizzas
    FOR loop = 0 TO noOfPizzas - 1
        t = chosenToppings[loop]
        IF t > 4 THEN
            t = 4
        END IF
        OUTPUT (loop + 1) & "> " & sizeNames[chosenSize[loop]] & " £" &
ToFixed(sizePrice[chosenSize[loop]], 2) & " + " & chosenToppings[loop] & "
toppings £" & ToFixed(toppingPrice[t], 2)
    NEXT loop
    OUTPUT "Subtotal: £" & ToFixed(totals[0], 2)
    OUTPUT "Discount: £" & ToFixed(totals[1], 2)
    OUTPUT "Delivery: £" & ToFixed(totals[2], 2)
    OUTPUT "Total: £" & ToFixed(totals[3], 2)
END SUBROUTINE
```

Assumptions

This table records each decision taken where the brief is silent, in the same way as the Supercar solution (Section 1.5).

The brief does not say…	Decision in this design
the most toppings allowed on one pizza	Any whole number from 0 up is accepted. 4 or more are all charged £2.50. A negative number is rejected.
whether money is shown to 2 decimal places	All money is shown to 2 decimal places using ToFixed, as in the Supercar solution.
whether an order of exactly £20.00 gets the discount	No. The brief says “over £20”, so £20.00 exactly gets no discount.
how the size is chosen	Each size has an ID from 0 to 2, shown on the menu. The ID matches its position in the arrays.
how delivery is entered	Staff type Y or N. Anything else is rejected.
whether the same size can be chosen twice	Yes. An order can have several pizzas of the same size.

Test plan design

The expected results were worked out by hand before the build. Nah runs these tests on Sorena's Flowgorithm program and records the actual results.

#	Type	Test data	Expected result	Actual result	Pass / Fail
1	Normal	2 pizzas: Large + 2 toppings, Medium + 4 toppings. Delivery Y.	Subtotal £16.50, discount £0.00, delivery £2.50, total £19.00		
2	Normal	3 pizzas: Large + 1 topping each. Delivery Y.	Subtotal £23.70, discount £2.37, delivery £2.50, total £23.83		
3	Normal	Bill shows the customer details and number of pizzas	Name, address, phone and "Number of pizzas" on the bill		
4	Boundary	Smallest order: 1 Small, 0 toppings. Delivery N.	Accepted. Total £3.25		
5	Boundary	Largest order: 6 Large, 9 toppings each. Delivery Y.	Accepted. Each pizza £9.65. Subtotal £57.90, discount £5.79, delivery £2.50, total £54.61		
6	Boundary	7 pizzas, then 1	"Error, must be between 1 and 6!" and asked again		
7	Boundary	0 pizzas, then 1	"Error, must be between 1 and 6!" and asked again		
8	Boundary	Subtotal exactly £20.00: 2 Small + 0 toppings, 1 Medium + 0 toppings, 1 Medium + 4 toppings. Delivery N.	No discount. Total £20.00		
9	Boundary	Subtotal just over £20: 1 Small + 0 toppings, 1 Large + 0 toppings, 1 Large + 4 toppings. Delivery N.	Subtotal £20.05, discount £2.01, total £18.04 (checks rounding)		
10	Boundary	4 toppings, then 5 toppings on a Small	Both charged £2.50 for toppings		
11	Boundary	Size ID 3, then 2	"Invalid choice. Please enter an ID from 0 to 2" and asked again		

#	Type	Test data	Expected result	Actual result	Pass / Fail
12	Boundary	Size ID −1, then 0	“Invalid choice. Please enter an ID from 0 to 2” and asked again		
13	Abnormal	Toppings −1, then 0	“Invalid. Toppings cannot be less than 0” and asked again		
14	Abnormal	Delivery “X”, then “N”	“Please enter Y or N” and asked again		
15	Abnormal	Blank customer name, then a name	“This cannot be blank, try again!” and asked again		
16	Abnormal	Letters for number of pizzas (“abc”), then 1	Run in Flowgorithm and record what happens		
17	Abnormal	Lower-case “y” for delivery	Run in Flowgorithm and record what happens		
18	Normal	1 Medium + 3 toppings, 1 Small + 1 topping. Delivery N.	Medium £5.50 + toppings £2.00, Small £3.25 + topping £0.75. Subtotal £11.50, no discount, total £11.50		

Nah fills in the last two columns when she runs each test, and adds a screenshot showing the input typed and the output on screen.

Worked calculation (Test 2)

The expected result for Test 2 was worked out by hand before the build. Nah compares the program's answer against it.

Step	Working	Result
1. One pizza	Large £7.15 + 1 topping £0.75	£7.90
2. Subtotal	3 pizzas × £7.90	£23.70
3. Discount check	£23.70 is over £20, so the 10% discount applies	Yes
4. Discount	£23.70 × 0.10	£2.37
5. After discount	£23.70 − £2.37	£21.33
6. Delivery	Delivery requested	£2.50
7. Final total	£21.33 + £2.50	£23.83

Requirements coverage

ID	Where it is handled	Tested by
R1	getCustomerDetails, checkBlank	3, 15
R2	getCustomerDetails, checkBlank	3
R3	getCustomerDetails, checkBlank	3
R4	getPizzas	1, 2, 4, 5
R5	choosePizzas, displayMenu, makeSizeChoice	1, 2, 11, 12
R6	getToppings	1, 2, 10, 13, 18
R7	getDelivery	1, 4, 14, 17
R8	getPizzas	4 to 7, 16
R9	populateMenu	4, 18
R10	populateMenu	1, 18
R11	populateMenu	1, 2, 5
R12	populateMenu, calcTotals	2, 18
R13	populateMenu, calcTotals	1
R14	populateMenu, calcTotals	18
R15	populateMenu, calcTotals	1, 5, 10
R16	calcTotals	2, 5, 8, 9
R17	calcTotals	1, 2, 5
R18	calcTotals, displayBill	1, 2, 5, 18
R19	calcTotals	1, 2, 8, 9
R20	calcTotals	2, 5, 9
R21	calcTotals	1, 2, 4
R22	calcTotals	1, 2, 4, 5
R23	displayBill	3
R24	displayBill	1, 2
R25	displayBill	1, 18
R26	displayBill	1, 2
R27	displayBill	2, 9
R28	displayBill	1, 2
R29	displayBill	1, 2